Lesson plan · for the teacher

Solving school problems in Python: unit check

Optimization · Level 3 · Unit check

Time	Materials
35 min	Board, chalk, paper. No devices needed.

Learning goals

- A timetable that works
- Exams without clashes
- The school bus route
- Matching teachers and classes

Activity: Model it in teams

1. Warm-up (5 min): For each of the four lessons, the class names one real decision at their school: a timetable, exam week, a route, sharing out jobs.
2. Model (15 min): Each team takes one decision and writes, on paper, its options (variables), its rules (constraints) and its goal, as plainly as they can, with a line of Python where they can.
3. Swap (10 min): Teams swap papers. Each team looks for one missing rule, the kind a solver would never guess, and writes it down.
4. Wrap-up (5 min): For each model, ask who the best plan might be hardest on, and what rule or second goal would protect them.

No devices needed. If you have a phone, run one team's model in the "Model it with CP-SAT" lab at the end.

Discussion

- Which school decision would you model first, and why?
- When is a quick rule good enough?
- Who should see a model's rules before it decides?

Exit ticket

- One school decision I will model, its goal, and one fairness rule.

Try it in class tomorrow

Pick one decision at your school this term (duties, exam slots, the bus, who teaches what). With two colleagues, write its options, rules and goal, model the smallest version in the "Model it with CP-SAT" lab, and show the result to the people it affects.

Worksheet · for pupils

Solving school problems in Python: unit check

Optimization · Level 3 · Unit check

Name _____ Class _____ Date _____

1 **In a CP-SAT timetable model, what is each lesson's variable?**

◯ The period it goes in ◯ Its teacher's name ◯ The number of pupils in the class

2 **INFEASIBLE means the solver ran out of time before finding a timetable.**

◯ True ◯ False

3 **The second rule should stop a teacher (one per subject) being in two classes at once, but timetables still clash. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  for c in classes:
2      m.add_all_different([p[c, s] for s in subjects])
3  for s in subjects:
4      m.add_all_different([p[c, s] for s in subjects])
```

4 **Match each school problem to the kind of problem it is.**

Exam slots without clashes	• • • •	An all-different rule
The bus visiting every village	• • • •	Assignment
One teacher for each class	• • • •	Travelling salesperson
No class with two lessons at once	• • • •	Graph colouring

5 **Which of these prove that 3 exam slots is the fewest possible? Choose all that apply.**

☐ Three exams that all clash with each other

☐ The solver says OPTIMAL when minimising the slots ☐ Greedy colouring used 4 slots

☐ DSatur found a plan with 3 slots